Event Sourcing in Production

Borjan Tchakaloff

He/Him

🌐 steadymonkey.eu

📣 @docbibi@freiburg.social

EuroPython 2024

Goals

Make you consider using Event Sourcing in Python, by:

- Showcasing head scratchers and how we solved them
- Promoting the `eventsourcing` library as a flexible solution in Python
- *Not running a tutorial* but showing some grounded examples
- *Not staring an academic discussion* but presenting a pragmatic report

1. (Illustrated) Concepts

2. Projection Evolution

3. Domain Evolution

4. Runtime Evolution

The example: Meetings

Or, a tool to organise meet-ups

- Simple meeting app for friends
  - A familiar concept: evening talks, game nights, week-end hikes, …
  - No web-scale problem, it's only for "us"
- Initial requirements: relaxed
  - Small load, worst case is short delay
  - Synchronous processing, no background worker

Event Sourcing, briefly

> Event Sourcing is that we will store facts which have occurred.
> Any state that we have is derived off of this set of facts.
> Any state that we have is transient.
>
> — *Greg Young, (2022)* [Micro-blogging about an upcoming book]

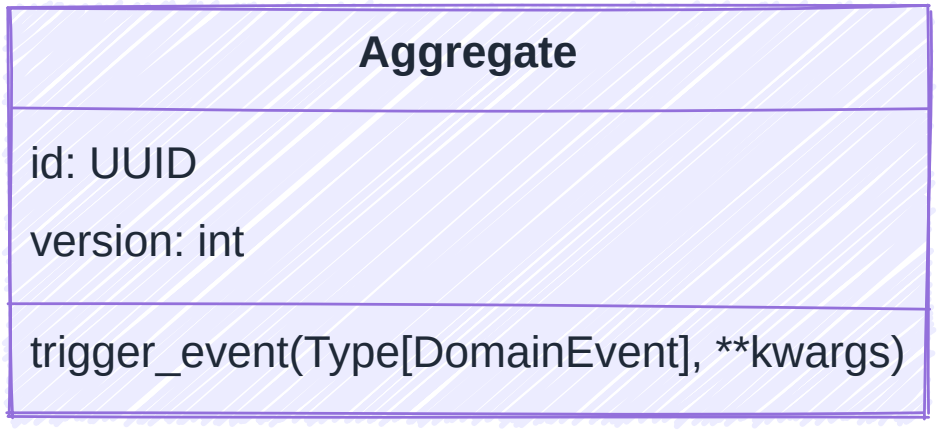
- Evolution of state over time; changes recorded as **Events**
  - State is (re)built by **replaying** events in sequence
- Vs. traditional approach persisting current state
- Very old concept, works well with our brains
  - Mesopotamia, 3000 BC, first record of market ledgers in clay tablets

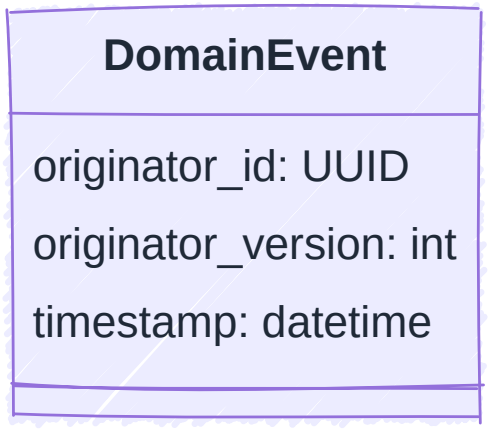
Domain-Driven Design (DDD), relevant ideas

> Domain-driven design is both a way of thinking and a set of priorities, aimed at accelerating software projects that have to deal with complicated domains.
>
> — *Eric Evans, in Domain-Driven Design (2003)*

- **Living understanding** shared by practitioners and modellers
- **Domain**: "A sphere of knowledge, influence, or activity."
- **Aggregates**: gatekeepers of the application state
  - Just enough logic to codify business rules (the domain)
  - About maintaining invariants in the face of data changes

`eventsourcing` library: domain






```
UserId = NewType("UserId", UUID)


class Meeting(Aggregate):
    class Created(DomainEvent):
        by_whom: UserId
        topic: str
        schedule: datetime

    class Edited(DomainEvent):
        topic: str

    class Rescheduled(DomainEvent):
        schedule: datetime
        reason: str

    ...  # Cancelled(reason),
         # ParticipationRegistered(who),
         # ParticipationWithdrawn(who)
```

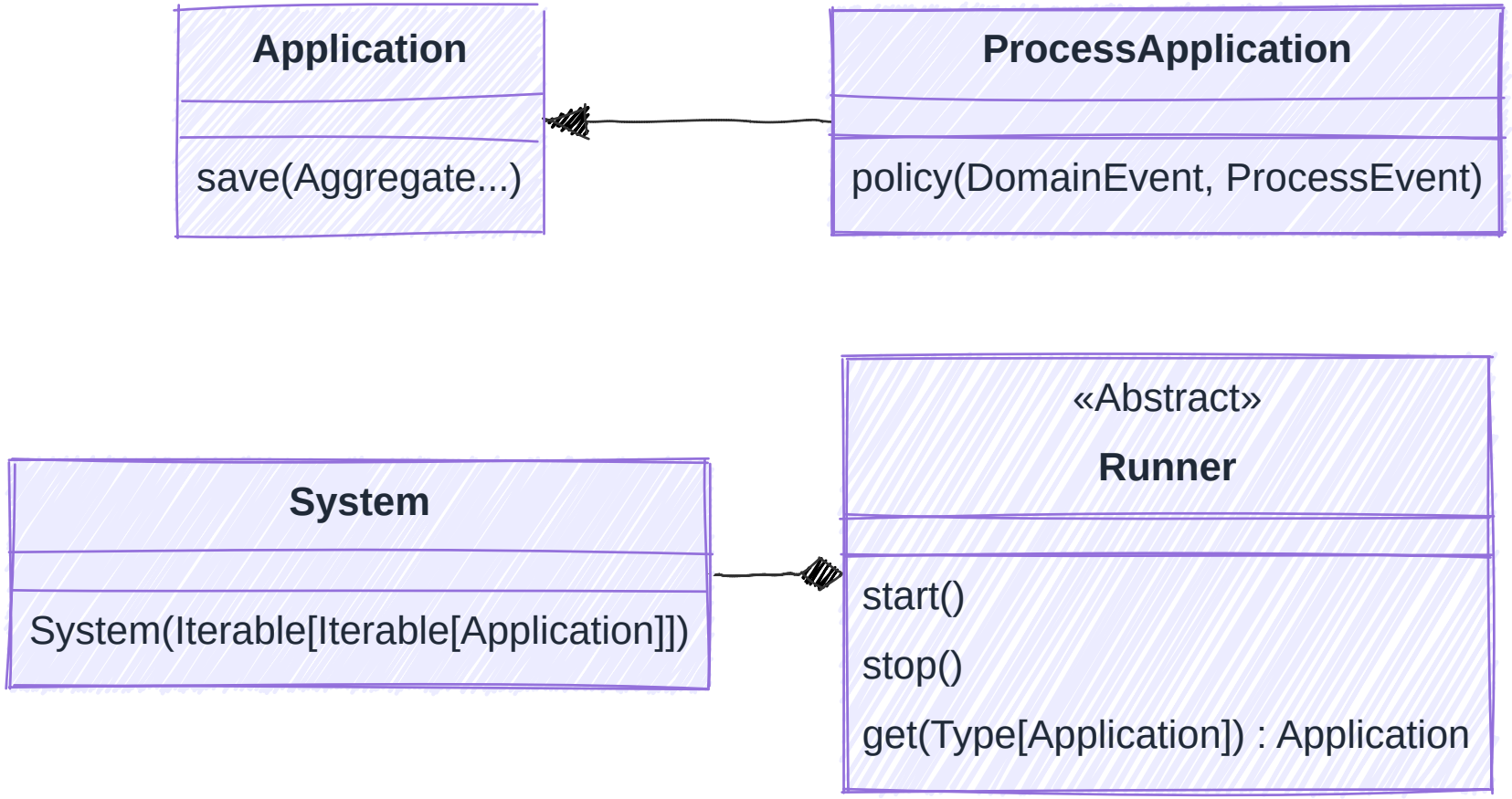
```
class Meeting(Aggregate):
    ...

    topic: str                      # set with `Created`, `Edited`
    schedule: datetime              # set with `Created`, `Rescheduled`
    is_cancelled: bool              # set with `Created`, `Cancelled`
    participants: set[UserId]       # set with `Created`, `ParticipationRegistered`,
                                    #  `ParticipationWithdrawn`
    # `by_whom` & `reason`s aren't needed for invariants nor idempotency (yet?)

    @event(Created)                 # shortcut binding event and method two-way
    def __init__(self, by_whom: UserId, topic: str, schedule: datetime):
        if not topic:               # invariant: topic isn't empty
            raise MeetingTopicIsRequired()
        self.topic = topic
        self.schedule = schedule
        self.is_cancelled = False
        self.participants = set()

    # Meeting(...) triggers Meeting.Created(...)
    # [Meeting.Created(...), ...] ⇒ Meeting(...)
```

Command Query Responsibility Segregation (CQRS), a simple pattern

> [CQRS] is the notion that you can use a different model to update information than the model you use to read information.
>
> — *Martin Fowler, in CQRS (bliki) (2011)*

- **Commands** mutate state and don't return domain information
- **Queries** return (a view of) the state without side-effects
- Commands and Queries are **different "things"** (models, objects, APIs)
  - I.e. targeted action (commands) vs. broad vision (queries)
  - Act on different data structures: write-models vs. read-models

`eventsourcing` library: processing & runtime



Meetings, the commands API

```
class Meetings(Application):
    def create(self, by_user_id: UUID, what: str, when: datetime) -> UUID:
        meeting = Meeting(UserId(by_user_id), what, when)
        self.save(meeting)          # no unique constraint other than unique identifier
        return meeting.id

    def edit(self, meeting_id: UUID, what: str):
        meeting = self.repository.get(meeting_id)
        meeting.edit(name)          # won't trigger any event in case of idempotency
        self.save(meeting)

    def reschedule(self, meeting_id: UUID, when: datetime, why: str): ...
    def cancel(self, meeting_id: UUID, why: str): ...
    def register_participation(self, meeting_id: UUID, for_user_id: str): ...
    def withdraw_participation(self, meeting_id: UUID, for_user_id: str): ...
```

Meetings, some read models

```
class UserInfo:  # another domain
    id: UUID
    name: str


class MeetingInfo:
    id: UUID
    by_whom: UserInfo
    topic: str
    schedule: datetime
    participants: set[UserInfo]

def upcoming_meetings(
    after: date
) -> list[MeetingInfo]:
    ...
```

```
class Stats:
    number_of_meetings: int
    number_of_participants: int

class OrganiserStats:
    of: UserInfo
    all_time_stats: Stats
    per_year_stats: dict[int, Stats]

def organiser_stats(
    of_user: UUID
) -> OrganiserStats:
    ...
```

Chapter 2

Projection evolution

1. **New derivative**: Fresh projection for a fresh view
2. **Change of mind**: Revise an existing projection

`Chapter 2 → Part 1`

New derivative, fresh projection

- Independent queries ⇒ more read-models
  - And/or plug in an ORM to leverage advanced selection
- Downstream `ProcessApplication` with a polymorphic policy
  - Lenient (ignore unknown events): good for prototyping
  - Strict (raise on unknown events): good with high coverage

Projecting to a Django read-model

```
class MeetingInfoProjector(ProcessApplication):
    @singledispatchmethod
    def policy(self, domain_event, _):
        raise NotImplementedError()

    @policy.register
    def _meeting_created(self, event: Meeting.Created, _):
        MeetingInfo.objects.create(
            id=event.originator_id, by_whom_id=event.by_whom,
            topic=event.topic, schedule=event.schedule,
        )

    ...  # and more policies for more events e.g. `Cancelled` deletes the entry

system = System([
    [Meetings, MeetingInfoProjector], [Meetings, StatsProjector]
])
```

Chapter 2 → Part 2

Revising a projection

Read-model

a. **Delete all entries**; update schema

b. **In-place update**

- Eventual consistency
- Schema conflicts

c. **Create new entries**; new model

Projector

a. **New process application**

b. Reset cursor & **replay history**

→ Whenever possible, KISS: delete all entries & replay history while offline

`Chapter 3`

An evolution of the *Meetings* domain

1. **Domain extension**: Extending the circle (of users), location now matters
2. **Invariant restriction**: Giving them a name, making them unique

`Chapter 3 → Part 1`

Domain extension, additional attribute

"Location becomes relevant for a meeting. How do we model this?"

- Approach A: **upcasting** existing events with a new attribute
- Approach B: **new event** to hold the new attribute

A. Upcast the event(s) with a new attribute

- Derive more information from existing events
- New consumers get a value
- Up-to-date consumers: manual update required

→ Fine without consumers, or projectors only

→ Lack of semantics and persistence

⇒ Keep it for non-core data (e.g. metadata)

```
class Meeting(Aggregate):
    ...
    location: str

    class Created(DomainEvent):
        by_whom: UserId
        topic: str
        schedule: datetime
        location: str
        class_version = 2

        @staticmethod
        def upcast_v1_v2(state):
            # static value or based on v1's state
            state["location"] = ...

    def __init__(
        by_whom: str,
        topic: str,
        schedule: datetime,
        location: str,
    ):
        ...
```

B. Define a new event

- New event to bridge historical data
- Backwards compatible, all consumers get the data
  (once they handle the new event)

→ Consumers need to handle the event

→ Proper semantics and persistence

⇒ Use it for core (domain) data

```
class Meeting(Aggregate):
    ...
    location: str | None

    class Created(DomainEvent):
        ...
        location: str | None
        class_version = 2

        @staticmethod
        def upcast_v1_v2(state):
            # known default for older aggregates
            state["location"] = None

    class LocationChanged(DomainEvent):
        location: str

    def __init__(
        by_whom: str,
        topic: str,
        schedule: datetime,
        location: str | None,
    ):
        ...

    def locate(self, location: str): ...
```

B. Define a new event (cont.)

→ Meeting location is core data, we want a new event (`LocationChanged`)
→ Status quo: all existing aggregates lack a location

⇒ One-off "workflow" to locate the aggregates

- Approach B1: value can be **derived** within the aggregate
- Approach B2: value needs to be **defined externally**

B1. Derive the value from the aggregate state

- `derive_missing_location()`
- One-off application command…
  - to loop over all aggregates (triggers n≥0 events); or,
  - directed at a specific aggregate (triggers n∈[0;1] event)

→ Requires an aggregate index, either in the application or outside

```
class Meeting(Aggregate):
    ...

    def derive_missing_location(self):
        if self.location:
            return
        location = ...            # actual derivation
        self.locate(location)     # aggregate command
                                  #  to trigger event


class Meetings(Application):
    ...

    def derive_missing_location(
        self, meeting_id: UUID
    ):
        meeting = self.repository.get(meeting_id)
        meeting.derive_missing_location()
        self.save(meeting)
```

B2. Define the value with a command (external trigger)

- Multi-step approach
  (long-lived process, *aka* workflow)
- Invariant "pending" for a while

Example approach to fill the gaps:

1. Track aggregates missing the value
2. Request tracker to notify owners
   - Careful: eventual consistency!
3. Wait for backfill / time-out
   - Archive or soft-delete

```
class Owner:
    id: UUID
    email: str

class MeetingMissingLocation:
    id: UUID
    owner_id: UUID


class TrackerApp(ProcessApplication):
    def policy(self, domain_event, _):
        ... # project and persist user info
            # project and persist meeting id
            # drop meeting on `LocationChanged`

    def notify_owners(self):
        ... # enqueue transactional emails
```

Chapter 3 → Part 2

Invariance restriction, unique values

→ No duplicates, no problem; otherwise requires a migration process
→ Not a technical problem, but a business one (related to invariant)

What to do about duplicates? Some approaches:

- **Mediatorial:** Open a long-lived process to resolve the conflicts
  - e.g. notify owners, give them a grace period, archive otherwise
- **Structural:** Merge affected aggregates (nest the other attributes)
  - e.g. one aggregate to rule them all (transition to a new type)
- **Temporal:** Combine affected aggregates (keep one, archive others)
  - e.g. most recently updated aggregates overwrites the older ones

```python
class MeetingTopic(Aggregate):
    topic: str
    winner: MeetingId | None
    losers: deque

    @staticmethod
    def create_id(topic: str) -> UUID:
        return uuid5(
            URL_NAMESPACE, f"/topics/{topic}"
        )

    @event("Created")
    def __init__(
        self, topic: str, meeting: MeetingId
    ):
        self.topic = topic
        self.winner = meeting
        self.losers = deque()

    @event("TakenOver")
    def _taken_over(self, meeting: MeetingId | None):
        if self.winner is not None:
            self.losers.append(self.winner)
        with contextlib.suppress(ValueError):
            self.losers.remove(meeting)
        self.winner = meeting
```

```python
    def take_control(self, meeting: MeetingId):
        if self.winner == meeting:
            return
        self._taken_over(meeting)

    def release_control(self, meeting: MeetingId):
        if self.winner != meeting:
            return
        try:
            new_winner = self.losers.pop()
        except IndexError:
            new_winner = None
        self._taken_over(new_winner)


class TemporalTracker(ProcessApplication):
    @singledispatchmethod
    def policy(self, domain_event, _):
        ... # Meeting.Created: take control
            # Meeting.LocationChanged: release control
            #   of previous location + take control
```

Chapter 4

Runtime evolution

1. **Synchronous systems**: Transactional vs. Sequential
2. **Asynchronous systems**: Multi-threading vs. Multi-processing
3. **Hybrid system**

Chapter 4 → Part 1

Synchronous systems

Transactional Event Sourcing

- Transaction boundary: system
- All-or-nothing: projection failures bubble up to domain

Sequential Event Sourcing

- Transaction boundary: domain
- If command returns, then the system is **consistent**
  - Else, processing will be retried next time (e.g. catch-up on startup)

Chapter 4 → Part 2

Asynchronous systems

Multi-threading

- Transaction boundary: domain
- One processing thread per domain

Multi-processing

- Transaction boundary: domain
- One process per domain

⇒ The system is **eventually consistent** (projected)

Chapter 4 → Part 3

Hybrid system

- Transaction boundary: domain
- Synchronous requests, in parallel

⇒ **Sequential processing × multiple workers** (threads or processes)

- The system is **consistently projected** after each triggering command
- Combine with e.g. `gunicorn --worker $(nproc --all)`

Chapter 4

Note on concurrency

- Higher concurrency ⇒ higher probability of conflicts
- `eventsourcing` relies on optimistic concurrency
  - Unique together constraint on ⟨id, version⟩ per domain

Chapter 4

Summary

	Transactional	Sequential	Asynchronous	Hybrid
Transactions	single	multiple	multiple	multiple
Consistency	direct	direct	eventual	direct
Concurrency	n/a	n/a	optimistic	optimistic
Recovery	n/a	manual	restart	auto restart
Performance	low	low	high	high

Alternatives

- Micro-Services Architecture: one service per domain
- Sharding of domain processing: distribute aggregate commands

Closing thoughts

> In a chaotic situation, options are better than things, so create options in the face of uncertainty.
>
> — *Kent Beck, Tidy First? (2023)*

- Event Sourcing's main strength is lossless evolution
  - Be it about domain (understanding), requirements, or scaling
- `eventsourcing` library: a flexible and powerful library

Extras

Personal Identifiable Information (PII)

PII in events? GDPR et al.!

- Python's "private" scope: cheap but only a convention;
  especially a problem in immutable logs
- Concretely: need for a separate stream that can be "forgotten" (deleted)
- Ideas that float around
  - Sensitive information encrypted at serialisation time
    - Maintain the private keys somewhere else
  - Independent store, keyed by agg. type / version / attribute
    - Pay attention to the consistency boundaries!
  - Complementary "private" agg. not visible on the application event stream

Two-step event triggering

```
class Meeting(Aggregate):
    ...

    def edit(self, topic: str):
        if self.is_cancelled:                # invariants
            raise MeetingIsCancelled()
        if not topic:
            raise MeetingTopicIsRequired()
        if self.topic == topic:              # idempotency
            return
        self._edited(topic=topic)

    @event(Edited)
    def _edited(self, topic: str):
        self.topic = topic
```

Internal index with an ES log

```
class Meeting(Aggregate):
    ...

    def locate(self, location: str):
        ...

    def derive_missing_location(self):
        if self.location:
            return
        location = ...      # derivation
        self.locate(location)


class MeetingLogged(DomainEvent):
    meeting_id: UUID
```

```
class Meetings(Application):
    ...
    _meeting_log: EventSourcedLog[
        MeetingLogged
    ]   # sequence of `MeetingLogged` events
        #  triggered by `Meetings.create()`

    def derive_missing_locations(self):
        meetings = [
            self.get(log_entry.meeting_id)
            for log_entry
            in self._meeting_log.get()
        ]
        for meeting in meetings:
            meeting.derive_missing_location()
        self.save(meetings)
```